



ITA602: Advanced Information Technology
Work Integrated Learning

Assessment 2
Project Action Plan

Orbis Scheduler

A Multi-Tenant Booking and Scheduling Platform for Service Based Businesses

Submitted To
Amr Van Den Adel

Submitted By
Hussain Jameel A00180177
Smitkumar Teli A00184801

Table of Contents

Table of Contents	2
1. Project Title	3
2. Project Goals (SMART)	3
3. Project Objectives	5
3.1 Deliverables	5
3.2 Milestones	5
4. Action Steps	6
4.1 Phase 1: Discovery and Scoping	6
4.2 Phase 2: System Design	7
4.3 Phase 3: Authentication and Tenant Isolation	9
4.4 Phase 4: Core Booking Backend	11
4.5 Phase 5: Frontend and Admin	12
4.6 Phase 6: Verification, Deployment and Submission	12
5. Tasks and Priorities	12
6. Roles and Responsibilities	14
6.1 Assignment of Action Items	14
7. Resource Allocation	15
8. Benchmarking and Evaluation	16
8.1 Comparison Against Competing Products	16
8.2 Evaluation Methodology	17
8.3 Benchmark Success Criteria	17
9. Ethics and Sustainability	18
9.1 Sustainability	18
10. Action Plan Timeline	19
11. Communication and Management Plan	20
11.1 Team Selection Criteria	20
11.2 Meetings, Agendas, Actions and Attendees	21
11.3 Stakeholder Reporting	21
12. References	22
Appendix A: Group Contract	23
Appendix B: Meeting Minutes and Agendas	24
Appendix C: Collaboration Evidence	26

1. Project Title

Orbis Scheduler: A Multi-Tenant Booking and Scheduling Platform for Service Based Businesses

This action plan carries forward the project reviewed in Assessment 1. Orbis Scheduler lets several independent service businesses run their appointment booking through one shared system while keeping each business's data strictly separated from every other tenant. Since submitting Assessment 1 we have moved from planning into delivery: the use case specification, the database design, the API design, the authentication module and the tenant isolation middleware are all built. This document sets out how the remaining work gets done, who does it, what it needs and how we will prove the finished product holds up against competing tools on the axis that matters most, secure multi-tenancy at a small business price point.

2. Project Goals (SMART)

Our overall goal is to deliver a working, deployed, security verified multi-tenant booking platform by the end of Week 12. To make that measurable rather than aspirational we have broken it into six SMART goals. Each one is specific about what gets built, measurable through an artifact or a test result, achievable within our two person capacity, relevant to the problem defined in Assessment 1 and bound to a date in the project timeline. Three of the six are already partly or fully banked, which gives us confidence the remaining three are realistic.

G1	Deliver a working multi-tenant booking platform (MVP) covering all fifteen use cases
Specific	Backend, frontend and admin dashboard built and deployed
Measurable	15 of 15 use cases pass functional testing
Achievable	Scoped for a two person team; stack proven in Weeks 1 to 5
Relevant	Directly addresses the SMB booking gap identified in Assessment 1
Time bound	Week 12 (17 Aug 2026)

G2	Enforce verifiable tenant isolation on every owner facing API route
Specific	UUID scoped middleware applied to all /api/owner routes
Measurable	Zero cross tenant data leaks across the 27 endpoint IDOR test suite
Achievable	Middleware pattern already implemented and unit checked
Relevant	Core differentiator against competing products (Section 8)
Time bound	Built Week 5; formally verified Week 10

G3	Complete the full backend API of 27 endpoints across seven groups before frontend work begins
Specific	Documented endpoint reference exists and is version controlled
Measurable	27 of 27 endpoints returning correct responses in Postman
Achievable	Sequenced backend first in the timeline to avoid rework
Relevant	Prevents frontend rebuilds against an unstable API
Time bound	End of Week 7 (19 Jul 2026)

G4	Ship a customer booking flow that requires no customer account
Specific	Public booking page, slot lookup and submission endpoints
Measurable	A test customer completes a booking end to end in under two minutes
Achievable	Request and approval model keeps the flow simple
Relevant	Removes friction, a key selling point for small businesses
Time bound	Week 8 (26 Jul 2026)

G5	Deploy the platform to production infrastructure
Specific	Vercel (frontend), Render (backend) and Neon (database)
Measurable	Live public URL accessible and serving the booking flow
Achievable	Free tiers confirmed sufficient for MVP scale
Relevant	Demonstrates real deployment practice, not just local work
Time bound	Week 10 (9 Aug 2026)

G6	Document the system to a professional standard
Specific	API reference, use case specification, ERD and code comments
Measurable	All four artifacts complete and consistent with the shipped code
Achievable	Three of four artifacts already delivered
Relevant	Supports assessment evidence and future maintainability
Time bound	Week 11 (16 Aug 2026)

3. Project Objectives

The goals above are reached through eight concrete deliverables checked off against seven dated milestones. The status column reflects where the project actually stands at the time of writing rather than a hypothetical plan: the design layer (D1 to D3) and the security foundation (D4, D5) are delivered, the core booking backend is in progress and the frontend, verification and submission phases remain ahead of us.

3.1 Deliverables

ID	Deliverable	Category	Status	Timing
D1	Use case specification (15 use cases across three actors)	Design	Completed	Week 3
D2	Entity relationship diagram and eight table PostgreSQL schema	Design	Completed	Week 4
D3	API endpoint reference (27 routes across seven groups)	Design	Completed	Week 4
D4	Authentication module (register, login, password reset, JWT issuance)	Backend	Completed	Week 5
D5	Tenant isolation middleware (JWT verification plus business scoping)	Backend	Completed	Week 5
D6	Core booking backend (availability, forms, slots, bookings)	Backend	In progress	Weeks 6 to 7
D7	Frontend application (owner dashboard, public booking page, admin dashboard)	Frontend	Upcoming	Weeks 8 to 9
D8	Deployed and security verified platform with final documentation	Delivery	Upcoming	Weeks 10 to 12

3.2 Milestones

ID	Milestone	Target date	Week	Status
M1	Project scope locked and fifteen use cases signed off	21 Jun 2026	Week 3	Achieved
M2	Design artifacts complete (ERD, schema, API reference)	28 Jun 2026	Week 4	Achieved
M3	Authentication and tenant isolation layer live and tested	5 Jul 2026	Week 5	Achieved
M4	Backend feature complete (all 27 endpoints working)	19 Jul 2026	Week 7	Upcoming
M5	Frontend complete and connected to the backend	2 Aug 2026	Week 9	Upcoming
M6	Security verification passed and platform deployed	9 Aug 2026	Week 10	Upcoming
M7	Final documentation and project submission	17 Aug 2026	Week 12	Upcoming

4. Action Steps

The work is organised into six phases. Each phase groups related tasks that together produce one or more of the deliverables above. The first three phases are complete, so for those we describe what was actually done and point to the artifacts as evidence. The remaining three describe the concrete steps ahead.

4.1 Phase 1: Discovery and Scoping

(Weeks 1 to 3, completed)

We locked the problem definition, the technology stack and the booking model in Week 1, then spent Weeks 2 and 3 turning the idea into a formal specification. The output is a use case document covering fifteen use cases across three actors: three customer use cases (initiating a booking, filling the business specific form, submission and pending review), seven business owner use cases (registration, password reset, reviewing bookings, availability setup, form building, dashboard management and retrieving the public link and widget snippet) and five administrator use cases (login, registration approval, platform overview, business listing and suspend or activate). Every use case includes a main flow, alternate paths and exceptions, which is what the remaining development phases are built against.

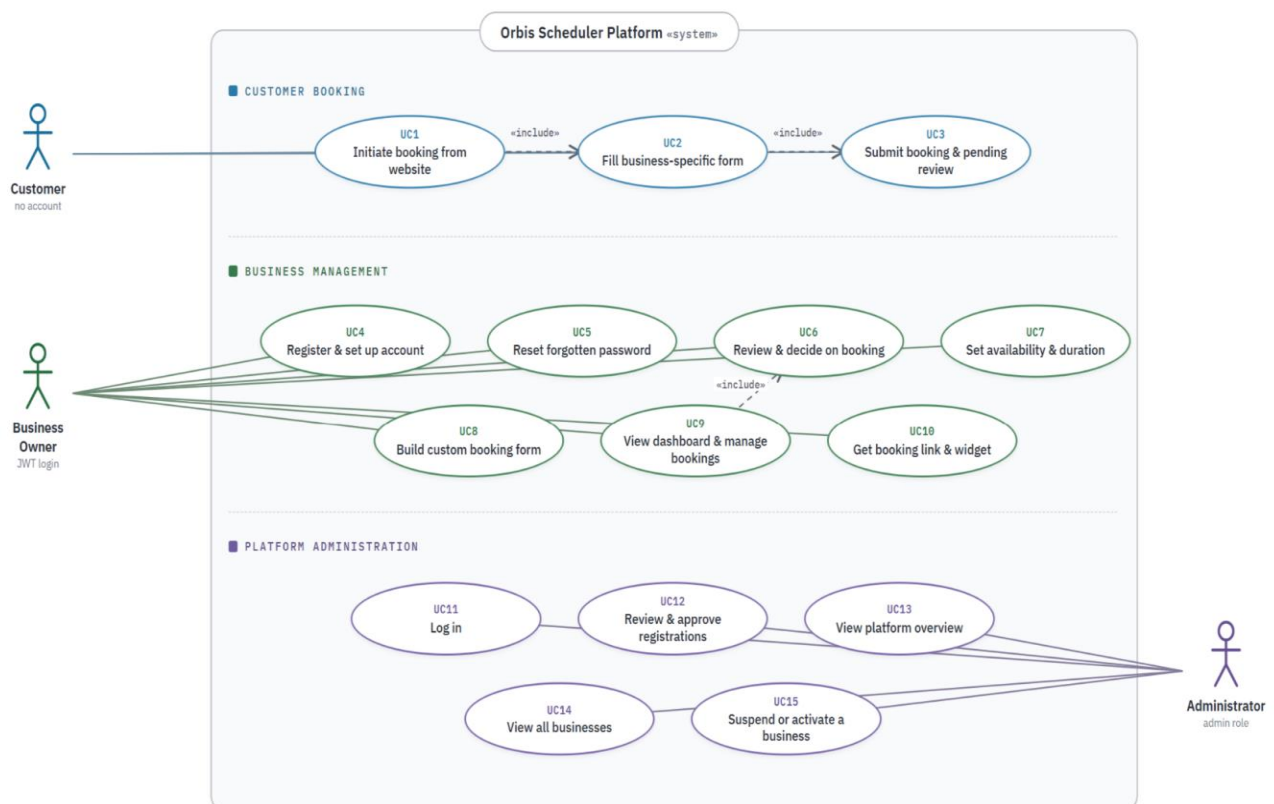


Figure 1. Orbis Scheduler use case diagram covering Customer, Business Owner and Platform Administrator.

4.2 Phase 2: System Design

(Week 4, completed)

With the behaviour specified, we designed the system that delivers it. Three artifacts came out of this phase. The first is the system architecture: a Next.js frontend deployed on Vercel talking to a Node.js and Express backend on Render, with PostgreSQL hosted on Neon accessed through Prisma 7. The API is split into three access tiers, public routes for customers, JWT protected owner routes and JWT protected admin routes, so the trust boundary is explicit in the URL structure itself.

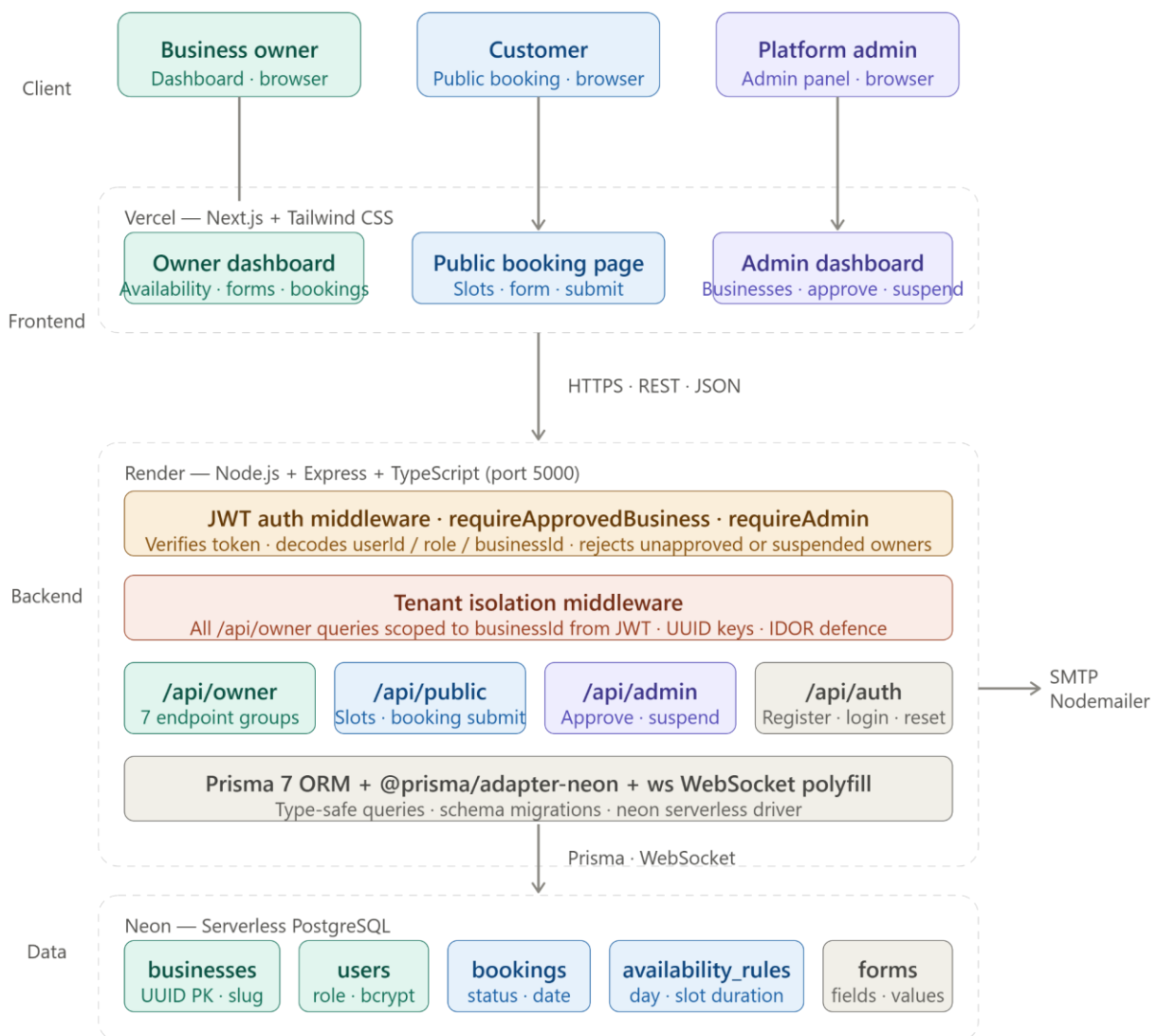


Figure 2. High level architecture showing the deployment targets and the three API access tiers.

The second artifact is the database design: an eight table relational schema where every tenant owned table carries the business identifier. The businesses table itself uses a UUID primary key so tenants cannot be discovered by stepping through sequential numbers. The bookings table deliberately carries a denormalised business_id so tenant scoped queries never need a join to establish ownership, which keeps the isolation check simple and fast.

Table	Purpose	Tenant linkage
users	Login accounts for business owners and platform admins; bcrypt password hashes; reset token fields	Owner or admin role
businesses	One row per tenant; UUID primary key used in every public URL; approval status workflow	UUID primary key
availability_rules	Weekly opening hours per business including breaks and slot duration	business_id (UUID)
booking_forms	The single active intake form per business with booking window setting	business_id (UUID)
form_fields	Dynamic fields on a form (text, textarea, dropdown, checkbox, radio) with JSONB options	Via form_id
bookings	Customer booking requests with status workflow (pending, approved, rejected, cancelled)	business_id (UUID, denormalised)
booking_field_values	Customer answers to each dynamic form field, stored per booking	Via booking_id



Figure 3. Entity relationship diagram for the seven table PostgreSQL schema.

The third artifact is the API reference: 27 endpoints across seven functional groups, each documented with its method, route, authentication requirement and request and response shapes before any endpoint code was written. Designing the full surface up front is what lets us mark backend feature completeness objectively at Milestone M4 rather than arguing about what done means.

Group	Prefix	Routes	Covers
Group 1 Auth	/api/auth	4	Owner registration, login, forgot and reset password
Group 2 Business profile	/api/owner, /api/public	3	Get and update own profile; load public booking page data
Group 3 Availability	/api/owner	2	Get and save the weekly availability schedule
Group 4 Forms and fields	/api/owner	6	Get and update form; add, edit, delete and reorder fields
Group 5 Slots	/api/public	1	Calculate available slots for a business on a given date
Group 6 Bookings	/api/public, /api/owner	6	Customer submits; owner lists, views, approves, rejects, cancels
Group 7 Admin	/api/admin	7	Platform stats; list, view, approve, reject, suspend, activate businesses

4.3 Phase 3: Authentication and Tenant Isolation

(Week 5, completed)

This phase built the security foundation everything else depends on. The registration endpoint validates input, rejects duplicate emails, hashes passwords with bcrypt at cost factor 12, generates a unique URL slug for the business and creates the user, the business (in pending approval state), a default booking form and its starter fields inside a single database transaction, so a half created tenant can never exist. The login endpoint returns an identical error for a wrong email and a wrong password so the response cannot be used to confirm which accounts exist, checks account and business state in a defined order and issues a JWT carrying the user id, the role and (for owners) the business id. Password reset follows the same information hiding principle: the response is deliberately identical whether or not the email exists, which prevents the endpoint being used to enumerate registered addresses.

The tenant isolation middleware then closes the loop. Every request to an owner route has its JWT verified, the decoded business id is attached to the request and every database query in the route handlers is scoped to that id. A separate requireAdmin middleware guards the admin tier and a requireApprovedBusiness check keeps owners whose registration is still pending or was rejected out of the dashboard. The practical effect is that a business owner is technically incapable of addressing another tenant's rows, even with a forged or edited request, because the scoping comes from the verified token rather than from anything the client sends.

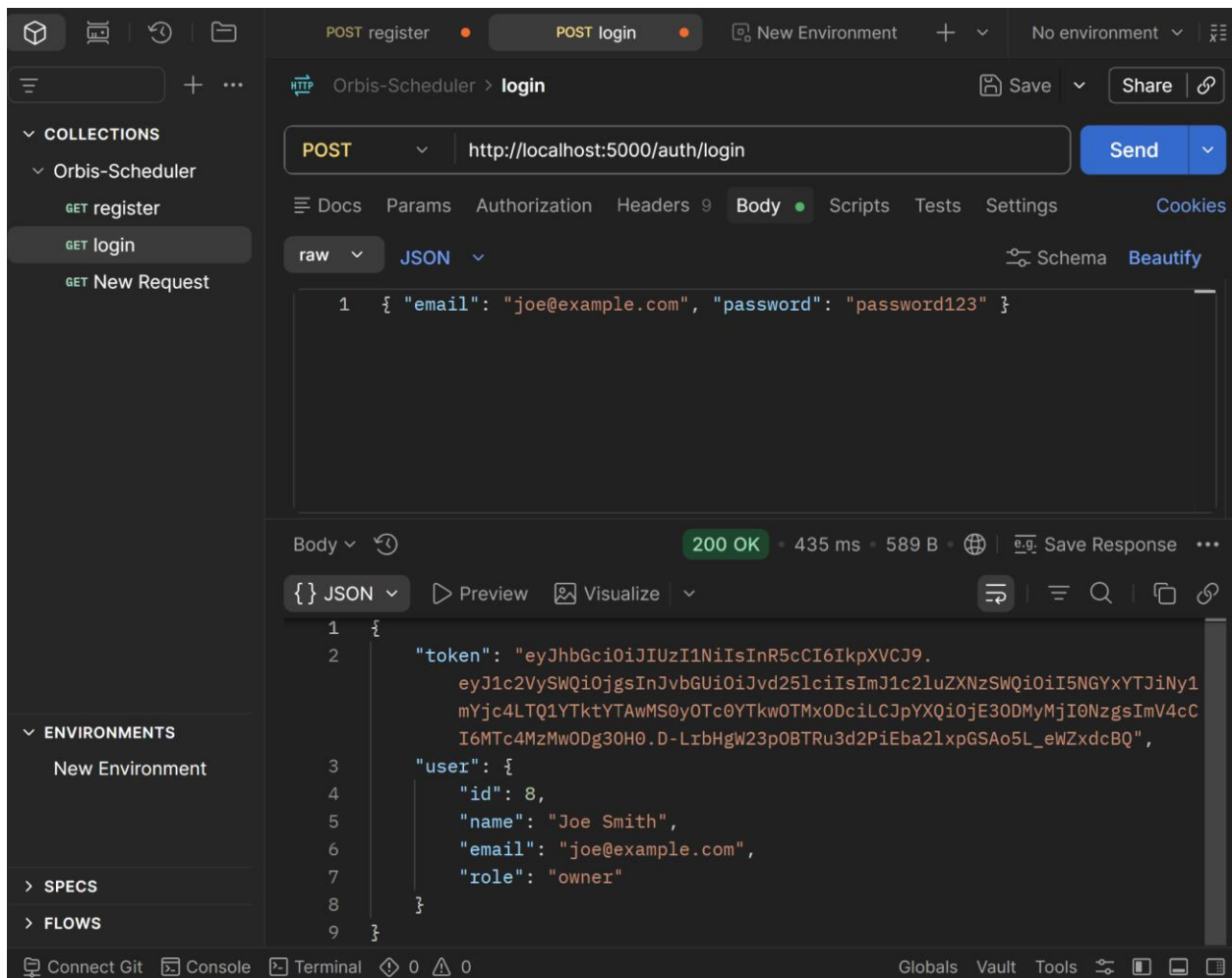


Figure 4. Verification evidence: valid owner authentication succeeding with status 200.

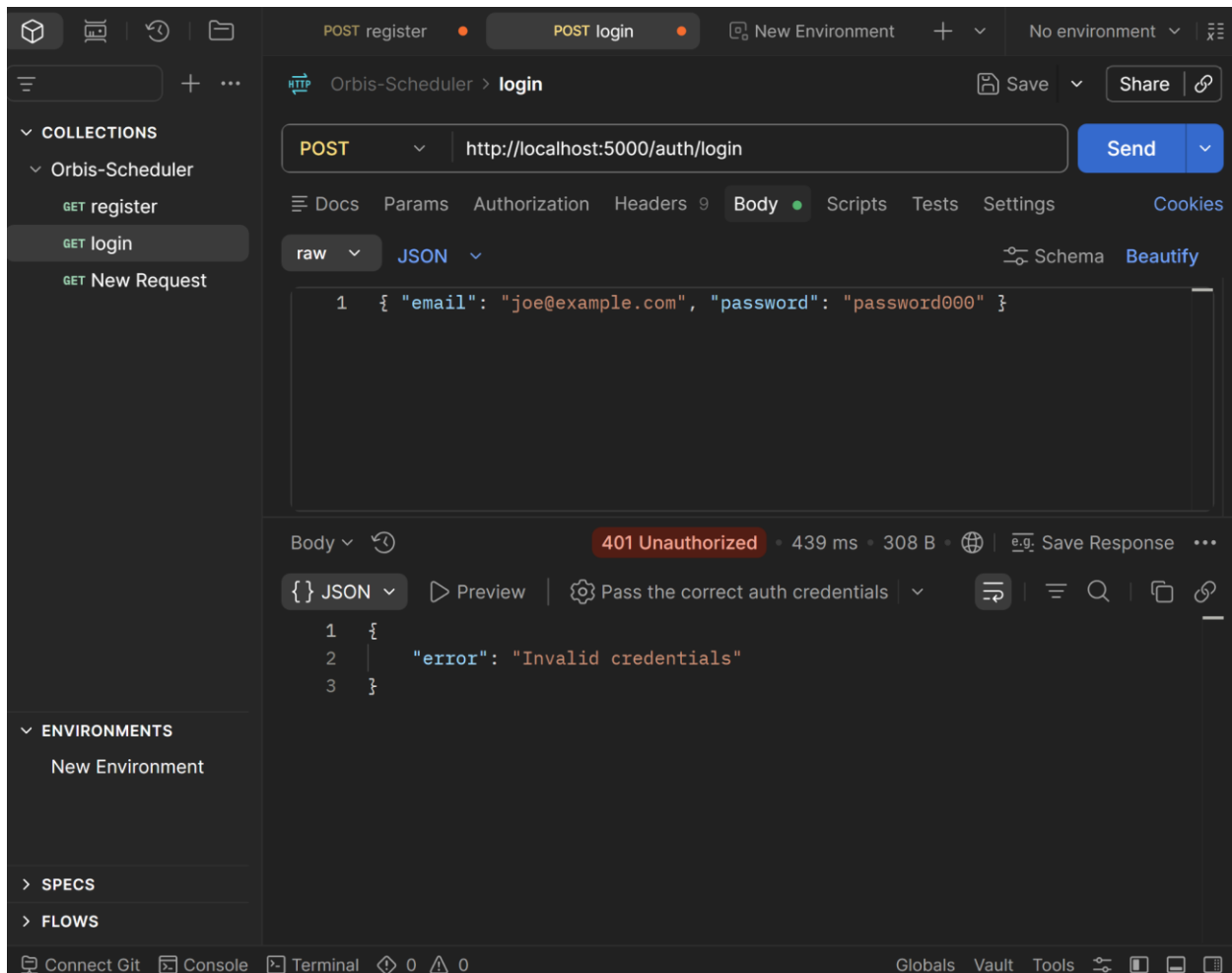


Figure 4.1. Verification evidence: Invalid owner authentication failed with status 401.

4.4 Phase 4: Core Booking Backend

(Weeks 6 to 7)

The next phase implements the remaining backend groups against the designed API surface. Week 6 covers the business profile endpoints, the weekly availability rules and the dynamic form builder (adding, editing, deleting and reordering fields). Week 7 covers the two pieces that make it a booking system. The first is the slot calculation logic, which computes available times on demand from the weekly rules, existing approved bookings and the business's booking window rather than pre generating rows. The second is the booking lifecycle, where a customer submission lands as pending and the owner approves, rejects or cancels it from the dashboard endpoints.

4.5 Phase 5: Frontend and Admin

(Weeks 8 to 9)

With the backend feature complete we build the interfaces on top of it. Week 8 scaffolds the Next.js application with Tailwind CSS, wires up the auth pages and delivers the two highest value screens: the owner dashboard with booking approval actions and the public booking page a customer walks through. Week 9 adds the admin dashboard (platform overview, business list, approve, suspend and activate), polishes the embeddable widget and finishes with a full functional pass of all fifteen use cases against the running system.

4.6 Phase 6: Verification, Deployment and Submission

(Weeks 10 to 12)

The final phase proves the system does what this plan claims. Week 10 runs the security verification described in Section 8 (the IDOR suite across all 27 endpoints, input validation checks and auth bypass attempts), fixes whatever surfaces and deploys the platform to Vercel and Render. Weeks 11 and 12 produce the final documentation, the API reference kept in the repository, code comments and the closing academic report before ending with submission on 17 Aug.

5. Tasks and Priorities

The phases above break down into fourteen tracked tasks. Each task lists its nested subtasks, the tasks it depends on and a priority: P1 tasks sit on the critical path and block other work, P2 tasks are core features that can flex within a week or two and P3 tasks are valuable but sacrificial if the schedule tightens. The dependency column is what drives the sequencing: nothing in T7 to T12 can start before T6 because every owner facing feature relies on the isolation middleware being in place, which is exactly why the security foundation was scheduled and completed first.

ID	Task	Nested subtasks	Depends	Priority	Status	Weeks
T1	Scope and use case definition	Actor mapping; 15 use case write ups; scope sign off	None	P1	Done	W1-3
T2	Database design	ERD; eight tables; enums; indexes; FK constraints	T1	P1	Done	W4
T3	API surface design	27 routes; auth tiers; request and response shapes	T2	P1	Done	W4
T4	Project scaffolding	Repo structure; env config; Prisma 7 with Neon adapter	T3	P1	Done	W5
T5	Authentication module	Register; login; JWT issuance; bcrypt; password reset; seed script	T4	P1	Done	W5
T6	Tenant isolation middleware	JWT verification; req.user typing; requireAdmin; requireApprovedBusiness	T5	P1	Done	W5

ID	Task	Nested subtasks	Depends	Priority	Status	Weeks
T7	Business profile endpoints	Get and update own profile; public page loader	T6	P2	Next	W6
T8	Availability endpoints	Weekly rules CRUD with validation	T6	P2	Next	W6
T9	Form builder endpoints	Form update; field add, edit, delete, reorder	T6	P2	Next	W6
T10	Slots and bookings	Slot calculation; submission; approve, reject, cancel	T7-T9	P1	Planned	W7
T11	Frontend build	Next.js scaffold; owner dashboard; public booking page	T10	P2	Planned	W8
T12	Admin dashboard	Stats; business list; approve; suspend; activate	T10	P3	Planned	W9
T13	Security verification	IDOR suite across 27 endpoints; input validation; auth bypass checks	T11	P1	Planned	W10
T14	Deployment and docs	Vercel and Render deploys; API docs; final report	T13	P1	Planned	W10-12

PRECEDENCE DIAGRAM METHOD · PDM

Orbis Scheduler

14 tasks · W1-W12

Critical path: T1-T6 → T7/T8/T9 → T10 → T11 → T13 → T14

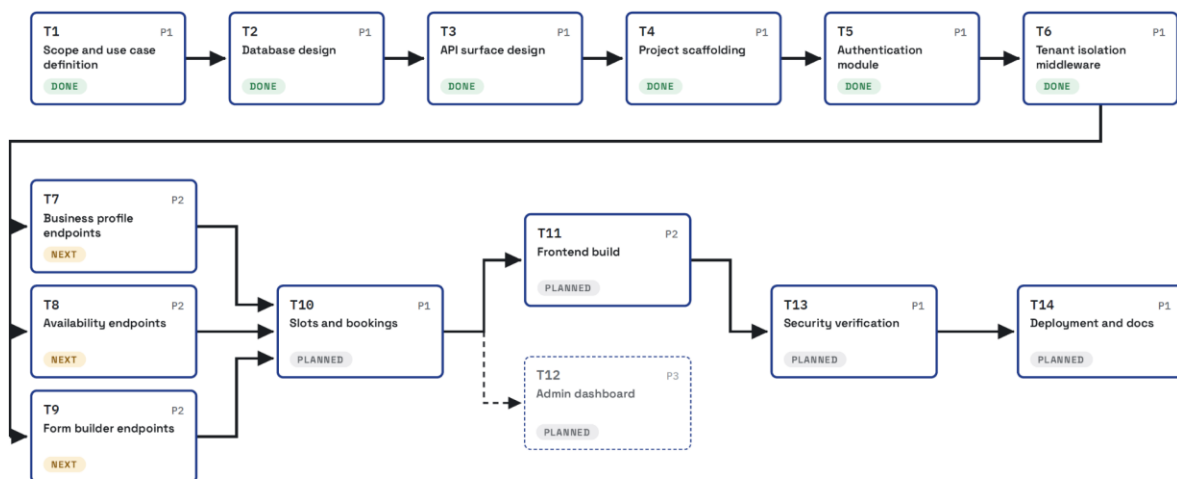


Figure 5. Precedence Diagramming Method (PDM)

The single riskiest task on the list is T10. Slot calculation touches availability rules, existing bookings and the booking window at once and it is the logic a customer directly experiences. It is scheduled with the full Week 7 to itself, immediately after its three dependencies land. It also carries P1 priority so that if anything slips it will be a P3 task (the admin dashboard) rather than the booking flow.

6. Roles and Responsibilities

We are a two person team working with a lead and support structure rather than an equal split of every task type, because that is what matches our actual skill profiles. Hussain works as a professional full stack developer and owns every architectural and engineering decision on the project. Smit is newer to development and works in a research, quality assurance and documentation role, picking up scoped action items assigned at the weekly sync and building experience through structured tasks reviewed by Hussain. This mirrors how a senior and junior pairing operates in industry and it has kept both of us productive without blocking on each other.

Member	Role	Responsibilities
Hussain Jameel	Project Lead, System Architect and Lead Engineer	Owns all architecture and technology decisions; designed the schema, ERD and API surface; implemented the authentication module and tenant isolation middleware; leads all remaining backend and frontend development; runs sprint planning and assigns weekly action items; final reviewer on every merge to main
Smitkumar Teli	Research, QA and Documentation Analyst	Works to weekly action items assigned at the Sunday sync; conducted the competitor and literature research feeding Sections 8 and 12; maintains meeting minutes and the communication log; formats and proofreads report submissions; executes the manual test checklists prepared for each sprint; captures the collaboration evidence (screenshots, logs) required for submission

6.1 Assignment of Action Items

Action items are assigned at the Sunday sync and tracked through the meeting minutes in Appendix B. The standing assignments are below; anything outside these is allocated by Hussain during sprint planning based on that week's phase.

Action item	Owner	How it runs
Architecture, schema, ERD, API design	Hussain	Sole owner; reviewed with Smit at the Week 4 sync before sign off
Auth module and middleware code	Hussain	Implemented and unit tested; Postman verification steps demonstrated to Smit
Remaining backend endpoints (T7 to T10)	Hussain	Built per the API reference; Smit runs the endpoint checklist after each group lands
Frontend and admin dashboard (T11, T12)	Hussain	Smit assists with copy text and click through testing
Competitor and literature research	Smit	Findings reviewed by Hussain before inclusion in this report
Meeting minutes and communication log	Smit	Compiled weekly; attached as Appendix B
Manual test execution (functional and IDOR checklists)	Smit	Checklists authored by Hussain; results logged by Smit

Action item	Owner	How it runs
Report formatting, references and submission	Smit	Final content review by Hussain before upload

7. Resource Allocation

The project runs on a deliberately zero dollar resource footprint. Every piece of infrastructure sits on a free tier (Neon for PostgreSQL, Render for the backend, Vercel for the frontend, GitHub for version control) and every tool in the workflow including VS Code, Postman, Figma, dbdiagram.io is free at our usage level. The scarce resource is therefore people time, roughly 8 to 10 hours per member per week as committed in the group contract. The table below allocates it per task alongside the software and infrastructure each task needs. Estimates for completed tasks are actuals; estimates for upcoming tasks are planned allocations that the timeline in Section 10 is built from.

Task	Software and infrastructure	Owner	Effort	Duration
Use case specification	Google Docs, draw.io	Hussain	10 h	Weeks 1-3 (done)
ERD and schema	dbdiagram.io, PostgreSQL	Hussain	6 h	Week 4 (done)
API surface design	Markdown docs, Postman	Hussain	5 h	Week 4 (done)
Auth module	Node.js, Express, TypeScript, Prisma 7, bcryptjs, JWT	Hussain	14 h	Week 5 (done)
Tenant middleware	Express middleware, Prisma, JWT	Hussain	6 h	Week 5 (done)
Competitor and literature research	Vendor docs, Google Scholar, OWASP	Smit	8 h	Weeks 2-4 (done)
Core backend endpoints	Node.js, Express, Prisma, Neon	Hussain	20 h	Weeks 6-7
Frontend application	Next.js, Tailwind CSS, Figma wireframes	Hussain	22 h	Weeks 8-9
Admin dashboard	Next.js, existing admin endpoints	Hussain	8 h	Week 9
Functional and IDOR testing	Postman collections, test checklists	Both	10 h	Week 10
Deployment	Vercel, Render, Neon (all free tier)	Hussain	5 h	Week 10
Documentation and final report	Word, GitHub docs folder	Both	12 h	Weeks 11-12

Two allocation decisions are worth calling out. First, the frontend carries the single largest block of hours even though the backend is the technically harder work, because interface work expands with polish and we would rather over allocate there than rush the screens a marker and a user actually see. Second, testing effort is shared: Hussain authors the checklists because he knows

where the system is most likely to break and Smit executes them, which doubles as an independent set of eyes on the results.

8. Benchmarking and Evaluation

An ICT innovation only counts as an innovation if it can be compared against what already exists and evaluated on its own merits. Our benchmarking approach has two halves: a comparison against competing products in the market, positioned on the axes the project was created for, plus an internal evaluation methodology that proves the finished system meets measurable criteria rather than just compiling.

8.1 Comparison Against Competing Products

The booking software market is crowded, so the honest comparison is not whether Orbis Scheduler has more features than Calendly, it does not and it is not trying to. The comparison is about the slot the incumbents leave open. Calendly and Setmore are excellent per user scheduling tools, but each business runs its own separate account and the platform's internal isolation model is not something an operator can inspect. Cal.com is open source and can be self hosted, but multi tenant operation sits behind its commercial platform plans. SimplyBook.me serves small businesses well as individual customers but genuine multi tenancy is an enterprise tier concern. None of them is built for the scenario Orbis targets: one operator (an agency, a co working community, a council business hub) hosting many independent small businesses on a single deployment where the tenant isolation model is explicit, documented and testable. The matrix below summarises the positioning; the assessments are based on each vendor's public documentation reviewed during our Week 2 to 4 research.

Capability	Orbis Scheduler	Calendly	Cal.com	SimplyBook.me	Setmore
True multi-tenant platform (many independent businesses on one deployment)	Yes	No (per user tool)	Partial (platform plans)	Partial (enterprise tier)	No
Tenant isolation model documented and testable	Yes	Not published	Partial (open source)	Not published	Not published
Custom per business intake form builder	Yes	Limited	Limited	Yes	Limited
Request and approval booking flow	Yes	Auto confirm focus	Auto confirm focus	Optional	Optional
No customer account required to book	Yes	Yes	Yes	Yes	Yes
Embeddable widget for the business website	Yes	Yes	Yes	Yes	Yes
Codebase owned and auditable by the operator	Yes	No	Yes (open source)	No	No
Cost at SMB scale	Free tier	Freemium per user	Free core	Subscription	Freemium

Read as a whole, the matrix shows Orbis Scheduler is not competing on feature breadth. It concedes maturity, integrations and polish to every product on that list. What it offers that they do not, at least not at a small business price, is a security first multi tenant core: UUID scoped tenants, token derived query scoping and an isolation model a developer can read, audit and attack test. That is the merit the evaluation below is designed to verify.

8.2 Evaluation Methodology

We evaluate the system on three fronts, loosely mapped to the quality characteristics of the ISO/IEC 25010 product quality model: functional suitability, security and performance efficiency, with a light usability check on top.

Functional evaluation uses the fifteen use cases as acceptance criteria. Each use case's main flow, alternate paths and exceptions are executed against the running system in Week 9 and again after security fixes in Week 10. A use case only passes when every documented path behaves as specified.

Security evaluation centres on a tenant isolation protocol we run against all 27 endpoints: authenticate as Business A, then request, modify and delete Business B's resources through every owner route, expecting an authorisation error on every single attempt. Around that core we run auth bypass checks (requests with missing, expired, tampered and wrong role tokens) and input validation checks (malformed payloads and injection style strings against every write endpoint), with each check mapped to the OWASP Top 10 category it exercises, primarily broken access control, identification and authentication failures and injection.

Performance evaluation is deliberately modest given free tier infrastructure: we record Postman response times for the slot calculation endpoint, the most computation heavy route in the system, then confirm it stays comfortably usable rather than claiming load tested throughput we cannot honestly measure. Usability is checked by timing a first time test user through the public booking flow.

8.3 Benchmark Success Criteria

Dimension	Measure	Pass threshold
Functional coverage	Use cases passing acceptance testing	15 of 15
Tenant isolation	Cross tenant leaks in the IDOR suite across all 27 endpoints	0
Authentication hardening	Auth bypass attempts succeeding (missing, expired, tampered, wrong role JWTs)	0
Input validation	Malformed or injection style payloads accepted by any endpoint	0
Booking flow usability	Time for a first time test customer to complete a booking	Under 2 minutes

Dimension	Measure	Pass threshold
Slot calculation responsiveness	Postman response time for GET /api/public/slots	Under 500 ms typical
Deployment	Platform reachable at a public URL with all flows working	Yes by Week 10

9. Ethics and Sustainability

The governance framework for this project is a four layer stack, with each layer answering a different question about how the platform handles the people and data it touches.

The legal layer is the Australian Privacy Principles. The two principles with the most direct design impact are APP 3, which limits collection to what is reasonably necessary, plus APP 11, which requires reasonable steps to secure the information held. Orbis Scheduler collects the minimum a booking genuinely needs (name, email, optional phone and the answers a business asks for on its own form) and holds no customer passwords at all because customers never create accounts.

The design layer is Privacy by Design, which we apply as an architectural habit rather than a slogan: privacy protective defaults (businesses start in a pending state until an admin approves them, forms start minimal), data minimisation baked into the schema and no collection of sensitive information categories anywhere in the product.

The technical layer is the OWASP Top 10, used as a running checklist rather than a one off reference. Broken access control is addressed by the tenant middleware and UUID identifiers. Injection is handled through Prisma's parameterised queries plus input validation on every write route. Identification and authentication failures are covered by bcrypt hashing at cost factor 12, 24 hour JWT expiry and deliberately vague password reset responses. The Week 10 security verification in Section 8 is what turns this layer from claims into evidence.

The professional layer is the ACS Code of Professional Conduct, which we treat as the standard for our own behaviour on the project: honesty about what the system can and cannot do (including the limitations we documented in Assessment 1), competence through building on established patterns rather than inventing our own cryptography or session handling and the primacy of the public interest in choosing to make security the project's core rather than an afterthought.

9.1 Sustainability

The sustainability case for Orbis Scheduler is structural rather than decorative. Multi tenancy is itself a resource efficiency measure: one deployment, one database and one set of compute serves many businesses that would otherwise each run (or buy) separate systems. The infrastructure choices push the same direction, Neon's serverless PostgreSQL scales to zero when idle and the free tier deployment targets share physical infrastructure across thousands of projects, so the platform's idle footprint is close to nil. Data minimisation reduces what has to be stored and secured over time and the product carries no advertising, tracking or engagement

mechanics, so it consumes no more attention or data than the booking task requires. On the societal side, the platform lowers the technology barrier for exactly the businesses least able to afford enterprise software, which is a small but real contribution to keeping local service businesses competitive online.

10. Action Plan Timeline

The full plan spans twelve weeks from 1 Jun to 17 Aug 2026, roughly 240 person hours across the two of us at the committed 8 to 10 hours per week each. The table below times every action step and milestone; the status column shows the plan is five weeks in with all three completed milestones hit on their target dates, which is the strongest evidence we can offer that the remaining seven weeks are estimated realistically rather than optimistically.

Week	Dates	Action items	Milestone	Status
W1	1-7 Jun	Scope decisions, project naming, stack confirmation, actor mapping		Done
W2	8-14 Jun	Assessment 1 drafting; competitor and literature research begins		Done
W3	15-21 Jun	Fifteen use cases finalised; scope locked; group contract signed	M1	Done
W4	22-28 Jun	ERD, eight table schema, 27 endpoint API reference designed and reviewed	M2	Done
W5	29 Jun-5 Jul	Repo scaffolding; auth module (register, login, reset, JWT, seed); tenant isolation middleware; Postman verification	M3	Done
W6	6-12 Jul	Business profile, availability and form builder endpoint groups		Next
W7	13-19 Jul	Slot calculation, booking submission and the approve, reject, cancel flow	M4	Planned
W8	20-26 Jul	Next.js scaffold, auth pages, owner dashboard, public booking page		Planned
W9	27 Jul-2 Aug	Admin dashboard, widget polish, full functional pass of all 15 use cases	M5	Planned
W10	3-9 Aug	IDOR and security verification, bug fixes, deployment to Vercel and Render	M6	Planned
W11	10-16 Aug	Final report, API documentation, code comments		Planned
W12	17 Aug	Final submission	M7	Planned

Orbis Scheduler — Gantt Chart

12 weeks · 1 Jun – 17 Aug 2026 · ITA602



Figure 5. Gantt view of the twelve week action plan with milestones M1 to M7.

11. Communication and Management Plan

11.1 Team Selection Criteria

The team was formed around complementary rather than identical skills. Hussain brings professional full stack development experience (Angular, React, Node.js and relational databases in industry roles), which the project needed for its architecture and engineering load. Smit brings research and documentation strengths and a security interest, which the project needed for its literature base, its evidence trail and its testing discipline. The selection criteria we applied to ourselves were simple: between the two members the team must cover engineering, research and quality assurance, both members must commit 8 to 10 hours per week in writing and both

must accept the lead and support structure described in Section 6 so that decision making stays fast.

11.2 Meetings, Agendas, Actions and Attendees

The standing meeting rhythm is below. Every Sunday sync runs to the same three part agenda (what was finished, what is blocking, what is next), both members attend, Smit takes minutes and every meeting ends with named action items carried into the following week. The full dated minutes for Weeks 1 to 5, including agendas, attendees, decisions and actions, are attached as Appendix B. Smaller decisions between meetings stay on the record through pull request comments and the WhatsApp group with a 48 hour resolution rule.

Meeting	When	Where	Purpose
Weekly team sync	Every Sunday 7:00 PM	Microsoft Teams	Progress review, task allocation, blocker resolution
Class sessions	Scheduled lecture times	Torrens online classes	Progress updates and facilitator questions
Ad hoc decisions	As needed, 48 h resolution rule	WhatsApp group	Small decisions that cannot wait for Sunday

11.3 Stakeholder Reporting

We report to three audiences through three channels matched to what each actually needs. The learning facilitator receives formal progress through the assessment submissions and can verify build progress at any time through the public GitHub repository, where every completed sprint is tagged (for example week5-complete) and the docs folder holds the living design artifacts. The team itself reports internally through the weekly minutes and a shared Google Drive folder holding every document version, which preserves an audit trail of how the plan evolved. Future stakeholders of the platform, the business owners, are represented for now by the use case specification and will be reported to through the deployed product itself and its documentation from Week 10 onward. Escalation is defined in the group contract: anything the two of us cannot resolve within 48 hours goes to the learning facilitator early rather than late.

12. References

Australian Computer Society. (2014). *ACS code of professional conduct*. <https://www.acs.org.au/>

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org/>

Cavoukian, A. (2009). Privacy by design: The 7 foundational principles. *Information and privacy commissioner of Ontario, Canada*, 5(2009), 12.

Chong, F., Carraro, G., & Wolter, R. (2006). Multi-tenant data architecture. *MSDN Library, Microsoft Corporation*, 14-30.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. University of California, Irvine.

Järvinen, H. M., & Kaivonen, C. L. S. (2024). GUIDELINES FOR SUSTAINABLE SOFTWARE.

Fontanarrosa, S. (2024). *Green Software Engineering*. Packt Publishing Ltd.

Iso, I. S. O. (2011). Iec25010: 2011 systems and software engineering—systems and software quality requirements and evaluation (square)—system and software quality models. *International Organization for Standardization*, 34(2910), 208.

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON web token (JWT) (RFC 7519). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc7519>

Krebs, R., Momm, C., & Kounev, S. (2012). Architectural concerns in multi-tenant saas applications. *Closer*, 12, 426-431.

Office of the Australian Information Commissioner. (n.d.). Australian Privacy Principles. <https://www.oaic.gov.au/privacy/australian-privacy-principles>

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum: The rules of the game. *Scrum. org*, 133-152.

OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/2021/>

Appendix A: Group Contract

The group contract for Assessments 1, 2 and 3, completed and signed by both members on 15 Jun 2026, is reproduced in full below as required by the assessment brief. The roles, responsibilities, ground rules, meeting schedule and time commitments recorded in the contract remain current; the only refinement since signing is the formalised action item assignment table in Section 6.1 of this report.

Group Member Contact Details

Name	Contact	Email
Hussain Jameel	0480 612 563	Hussain.Jameel@student.torrens.edu.au
Smitkumar Teli	0466 970 827	Smitkumar.Teli@student.torrens.edu.au

Ground Rules

1. All team members respond to messages within 24 hours and notify the group in advance if they are unavailable due to work, study or personal commitments.
2. All allocated tasks are completed by agreed deadlines. If a member cannot complete a task they inform the team immediately so alternative arrangements can be made.

Group Goals (in order of priority)

1. Successfully design and develop the Orbis Scheduler booking and scheduling platform according to the project proposal and assessment requirements.
2. Achieve a Distinction or High Distinction grade across Assessments 1, 2 and 3 through effective teamwork and quality documentation.
3. Develop practical experience in software engineering, cybersecurity, project management, cloud technologies, testing and team collaboration.

Agreed Time Commitment

Eight to ten hours per week per group member for the duration of the project.

Roles and Responsibilities

Member	Role	Responsibilities
Hussain Jameel	Project Lead, System Architect and Lead Engineer	Owns all architecture and technology decisions; designed the schema, ERD and API surface; implemented the authentication module and tenant isolation middleware; leads all remaining backend and frontend development; runs sprint planning and assigns weekly action items; final reviewer on every merge to main
Smitkumar Teli	Research, QA and Security and Documentation Analyst	Works to weekly action items assigned at the Sunday sync; conducted the competitor and literature research feeding Sections 8 and 12; maintains meeting minutes and the communication log; formats and proofreads report submissions; executes the manual test checklists prepared

		for each sprint; captures the collaboration evidence (screenshots, logs) required for submission
--	--	--

Meeting Schedule (in addition to meetings in class)

Weeks	When	Where	Preparation required
Weeks 1 to 12	Every Sunday, 7:00 PM	Microsoft Teams or WhatsApp call	Review completed tasks, identify issues and prepare weekly progress updates
Weeks 1 to 12	Scheduled class sessions	Torrens University online classes	Bring project documents, progress updates and discussion points

Decision Making

Major project decisions are discussed during the scheduled weekly meetings and made through mutual agreement. If agreement cannot be reached immediately, options are reviewed and a final decision is made within 48 hours. Urgent decisions may be resolved through WhatsApp or Microsoft Teams to prevent delays to project milestones.

Group Expectations

1. No group member will be excluded from group activities.
2. Late delivery and broken promises of delivery are not acceptable.
3. All group members will contribute to discussions.
4. Speak kindly, but honestly.
5. Offer solutions.

Declarations and Signatures of Group Members

Both members agree to the rules and conditions of this group contract for ITA602 Assessments 1, 2 and 3 and acknowledge that failing to adhere to them or to contribute equitably may result in a lower individual assessment mark, as set out in the contract's learning facilitator intervention clause.

Name	Student ID	Signature	Date
Hussain Jameel	A00180177	Hussain	21/06/2026
Smitkumar Teli	A00184801	Smit	21/06/2026

Appendix B: Meeting Minutes and Agendas

Minutes are recorded by Smitkumar at every Sunday sync and stored in the shared Google Drive folder. Each meeting held across Weeks 1 to 5 is recorded below in a standard format covering the agenda set before the meeting, the decisions made and the action items assigned.

Meeting 1	Week 1 team sync, Sunday 7 Jun 2026, 7:00 PM
Platform	Microsoft Teams
Attendees	Hussain Jameel, Smitkumar Teli
Recorded by	Smitkumar Teli
Agenda	Project naming; scope boundaries; technology stack; booking model
Decisions	Name locked as Orbis Scheduler. Stack locked: Next.js frontend, Node.js with Express and TypeScript backend, PostgreSQL on Neon, JWT authentication. Request and approval booking model chosen over auto confirmation.
Action items	Hussain to draft the use case skeleton and actor map. Smit to compile the competitor list and begin the literature review.

Meeting 2	Week 2 team sync, Sunday 14 Jun 2026, 7:00 PM
Platform	Microsoft Teams
Attendees	Hussain Jameel, Smitkumar Teli
Recorded by	Smitkumar Teli
Agenda	Assessment 1 draft review; section ownership; reference formatting
Decisions	Section owners confirmed. APA reference list to be rebuilt from original sources rather than citation generators.
Action items	Hussain to outline the system architecture. Smit to clean up the APA references and proofread the full draft.

Meeting 3	Week 3 team sync, Sunday 21 Jun 2026, 7:00 PM
Platform	Microsoft Teams
Attendees	Hussain Jameel, Smitkumar Teli
Recorded by	Smitkumar Teli
Agenda	Use case sign off; group contract; Week 4 design plan
Decisions	All fifteen use cases across three actors signed off. Group contract completed and signed by both members. ERD and API design scheduled for Week 4.
Action items	Hussain to build the ERD and draft the API endpoint reference. Smit to prepare the functional test checklist template.

Meeting 4	Week 4 team sync, Sunday 28 Jun 2026, 7:00 PM
Platform	Microsoft Teams
Attendees	Hussain Jameel, Smitkumar Teli
Recorded by	Smitkumar Teli

Agenda	ERD and API reference review; auth build plan
Decisions	Eight table schema approved, including UUID keys for businesses and a denormalised business_id on bookings for direct tenant queries. All 27 endpoints across seven groups signed off. Auth module scheduled first with the middleware immediately after.
Action items	Hussain to implement registration, login, password reset and the seed script. Smit to draft the IDOR test checklist against the endpoint list.

Meeting 5	Week 5 team sync, Sunday 5 Jul 2026, 7:00 PM
Platform	Microsoft Teams
Attendees	Hussain Jameel, Smitkumar Teli
Recorded by	Smitkumar Teli
Agenda	Middleware demonstration; Assessment 2 final review; Week 6 sprint plan
Decisions	Tenant isolation middleware demonstrated and accepted, with cross tenant requests rejected as designed. Assessment 2 report reviewed by both members and approved for submission. Week 6 sprint scoped to the business profile, availability and booking form endpoint groups.
Action items	Hussain to begin the business profile and availability endpoints. Smit to file the Week 5 test results in the shared drive and update the functional test checklist for Week 6.

Appendix C: Collaboration Evidence

The following GitHub repository attached contains all the resources including documentation and code as evidence of ongoing collaboration across the trimester.

Github Repository: <https://github.com/hussainjameel/orbis-scheduler>